Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Operating Systems

ENCS3390

Project Report

Prepared by:

Al-zahra Nassif 1220168

Instructor: **Yazan Abu Farha**

Section: **1**

Date: 8/12/2024

Abstract

This project explores parallel computing techniques for efficiently finding and list the top 10 most frequent words on a large dataset. The naive approach processes the dataset sequentially, while multiprocessing and multithreading distribute the workload across multiple processes or threads. Execution times are compared, and scalability is analyzed using Amdahl's law. The results will show that parallel computing reduces Execution time, so multiprocessing and multithreading will be much faster than the naive approach.

Table of Contents

Abstract I
Table of Contents II
Table of Figures III
List of Tables IV
1. Discussion & Analysis 1
1.1 My environment Description 1
1.2 The API and functions used for creation 1
1.3 Serial part of my code 3
1.4 Function FindFrequentWords 4
1.5 An analysis according to Amdahl's law 6
1.6 Performance Measurements 9
Conclusion 10

Table of Figures

Figure 1: Multithreading creation 2
Figure 2:multiprocessing creation. 3
Figure 3: read file function 3
Figure 4:Sorting & print the final results 4
Figure 5:Naive FindFrequentWords. 4
Figure 6: Multithreading FindFrequentWords dividing technique 5
Figure 7: multiprocessing FindFrequentWords dividing technique 5
Figure 8: Naïve Approach Results. 6
Figure 9:Multithreading Approach Results 7
Figure 10:Multiprocessing Approach Results 8

List of Tables

Table 1: Multithreading calculations 7
Table 2:Multiprocessing calculations 8
Table 3:Performance Measurements 9

1. Discussion & Analysis

1.1 My environment Description

My processor is 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz 2.80 GHz.

- Cores: 4 physical cores with 8 threads (logical).
- Speed: base speed of 2.8 GHz and boost up to 4.7 GHz.
- Memory (RAM): 8GB (7.79 GB usable).
- OS: Windows 11 64-bit.
- Programming Language: C.
- IDE tool: Ubuntu terminal.
- Virtual Machine: didn't use.

1.2 The API and functions used for creation

1.2.1 Multithreading Approach

To implement multithreading, I used the POSIX Threads (pthreads) API. At first, I include its library, and defined the number of threads so we can try different numbers of threads and compare the outcome.

```
#include <pthread.h> //For pthread_create(), pthread_join(), pthread_mutex_lock(), and pthread_mutex_unlock()
#define NUM_THREADS 8
```

And since the threads shared the same global array "wordcounted" I used a mutex for thread-safe updates on it

```
pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;
```

Functions:

- pthread_create(): create a new thread.
- pthread_join(): This function is used to wait for all threads to finish their execution. since we have more than one thread I put both in for loop up to NUM_THREADS-1 .
- pthread_mutex_lock(), and pthread_mutex_unlock(): used for thread synchronization, to ensure that just one thread can access the CS at time to avoid the race condition .

```
pthread_t threads[NUM_THREADS];

// Create threads
for (int i = 0; i < NUM_THREADS; i++) {
        int *threadNumber = malloc(sizeof(int));
        *threadNumber = i;
        pthread_create(&threads[i], NULL,  FindFrequentWords,  threadNumber);
}

// Wait for all threads to finish
for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
}
```

Figure 1: Multithreading creation

1.2.2 Multiprocessing Approach

To implement multiprocessing, I used Unix processes. Since the processes don't shares global variables as threads, I created a shared memory, so we also need synchronization to avoid race condition, I used a semaphore this time.

At first, I included the needed libraries

```
#include <sys/wait.h>  // For wait()
#include <unistd.h>    // For fork()
#include <sys/mman.h> // For mmap() , munmap()
#include <semaphore.h>  // Semaphore
```

Then I defined the semaphore

```
sem_t *semaphore;
```

Functions:

- fork():create new child processes.
- Wait(): This function is used by the parent process to wait for the child processes to finish execution before continuing.
- mmap(): create shared memory regions
- sem_post() & sem_wait(): for process synchronization, ensuring that only one process updates the shared word counts at a time

```c
int pid[ NUM_PROCESS];

// Loop to create multiple child processes
for (int i = 0; i < NUM_PROCESS; i++) {

        pid[i] = fork();  // Create a new child process

        if (pid[i] < 0) {  // Error handling
                printf("An error occurred with fork!\n");
                return 1;
        }
        else if (pid[i] == 0) {  // Child process
                FindFrequentWords(i);  // Call the function for processing
                exit(0);  // Exit child process after completing task
        }
}

// Parent process waits for all child processes to complete
for (int i = 0; i < NUM_PROCESS; i++) {
        wait(&pid[i]);  // Wait for any child process to finish
}
```

Figure 2:multiprocessing creation.

1.3 Serial part of my code

➢ The serial portion of the multiprocessing and multithreading code includes:

- **File reading**
  This function exists in the three approaches

```c
void readDataFile() {
        FILE *data = fopen("/home/alzahra/enwik8.txt.txt", "r");
        if (data == NULL) {
                printf("Error opening file, the path you entered is wrong ! \n");
                return;  // Exit the program if the file cannot be opened
        }

        char word[MAX_WORD_LENGTH];

        while (fscanf(data, "%99s", word) == 1) {
                snprintf(words[counter].word, MAX_WORD_LENGTH, "%s", word);
                words[counter].count = 1;
                counter++;
        }
        fclose(data);
}
```

Figure 3: read file function

- **Sorting & print the final results**
  The time complexity of this sort is O(nlogn)

```c
int compare_words(const void *a, const void *b) {
        struct singleword *wordA = (struct singleword *)a;
        struct singleword *wordB = (struct singleword *)b;
        return wordB->count - wordA->count;
}

void sortWords(struct singleword *words, int size) {
        qsort(words, size, sizeof(struct singleword), compare_words);
}

void printTopWords() {
        printf("Top 10 most frequent words:\n");
        for (int i = 0 ; i < 10  && i < realnum ; i++) {
                printf("%d)  %s - %d\n",(i+1), wordcounted[i].word, wordcounted[i].count);

        }
}
```

Figure 4:Sorting & print the final results

1.4 Function FindFrequentWords

This is the function that every thread or process take it in to execute it on parallel

- In Naïve approach

```c
void FindFrequentWords() {
        char wordToFound[MAX_WORD_LENGTH];
        for (int i = 0 ; i < counter; i++) {
                strcpy(wordToFound, words[i].word);
                int found = 0;

                // Try to find the word in wordcounted array
                for (int j = 0; j < realnum; j++) {
                        if (strcasecmp(wordcounted[j].word, wordToFound) == 0) {
                                // If found, update count
                                wordcounted[j].count++;
                                found = 1;
                                break;
                        }
                }

                // If word is not found, add it to wordcounted
                if (!found && realnum < counter) {
                        snprintf(wordcounted[realnum].word, MAX_WORD_LENGTH, "%s", wordToFound);
                        wordcounted[realnum].count = 1;
                        realnum++;

                }
    }
}
```

Figure 5:Naive FindFrequentWords.

- In multithreading approach

At the beginning of the function, I divided the total dataset (counter) into equal parts for each thread. Each thread is assigned a specific range of data based on its number (number). The sizePerThread determines how many items each thread processes. The startingIndex and endingIndex specify the range for each thread. If there are leftover items (when counter is not evenly divisible by NUM_THREADS), the last thread handles the remaining items

by adjusting its endingIndex to include all remaining elements. This ensures an even distribution of the workload across threads.

```
void*  FindFrequentWords(void* arg) {
        int number = *(int*)arg;  // Thread number
        // Divide the workload among threads
        int sizePerThread = counter / NUM_THREADS;
        int startingIndex = number * sizePerThread;
        int endingIndex = (number + 1) * sizePerThread;

        // Handle the last thread to cover any remaining words
        if (number == (NUM_THREADS - 1)) {
                endingIndex = counter;
        }

        char wordToFound[MAX_WORD_LENGTH];


        for (int i = startingIndex; i < endingIndex; i++) {
                strcpy(wordToFound, words[i].word);
                int found = 0;
```

Figure 6: Multithreading FindFrequentWords dividing technique

- In multiprocessing approach

  the dividing technique is same as multithreading.

```
void FindFrequentWords(int number) {

        int sizePerProcess = counter / NUM_PROCESS;
        int startingIndex = number * sizePerProcess;
        int endingIndex = (number + 1) * sizePerProcess;

        if (number == (NUM_PROCESS - 1)) {
                endingIndex = counter;
        }

        char wordToFound[MAX_WORD_LENGTH];

        for (int i = startingIndex; i < endingIndex; i++) {
                strcpy(wordToFound, words[i].word);
                int found = 0;
```

Figure 7: multiprocessing FindFrequentWords dividing technique

1.5 An analysis according to Amdahl's law

➢ **Naïve Approach**

```
alzahra@zahra:~$ ./naive
Top 10 most frequent words:
1)  the - 1061396
2)  of - 593677
3)  and - 416629
4)  one - 411764
5)  in - 372201
6)  a - 325873
7)  to - 316376
8)  zero - 264975
9)  nine - 250430
10)  two - 192644
In Naive system

Time taken to read Data file: 1.662161 seconds

Time taken to find the frequency of each word : 830.170953 seconds

Time taken to sort the top 10: 0.066957 seconds

Time taken to execute: 831.900145 seconds
```

Figure 8: Naïve Approach Results.

Serial Time = Reading File Time + Finding frequent words time+ sorting the top 10 words time

= 1.662161+830.170953+0.066957 = 831.900071 sec.

Serial Portion = (**Serial Time** /$\boldsymbol{Total\ Time}$(execution time)) ×**100**%

=(831.900071/831.900145)×100% = 99.99999% → No parallelization.

Maximum Speed up = $\mathbf{1}/(\boldsymbol{S} + ((\mathbf{1}-\boldsymbol{S})/\boldsymbol{N}))$

= 1 /(0.9999999+ (1−0.9999999)/ 1) ≈ 1

The analysis shows that adding more cores has no significant effect on performance, as 99.99999% of the execution time is spent on serial tasks that cannot be parallelized. Even with 8 processors, the maximum speedup remains essentially 1, indicating that the workload is largely unaffected by the number of cores, making parallelization ineffective in this case.

➢ **Multithreading Approach**

Serial Time = **Reading File Time** + **sorting the top 10 words time**

- *Note that Finding frequent words is not serial anymore*

Serial Portion = (**Serial Time** /$\boldsymbol{Total\ Time}$(execution time)) ×**100**%

Maximum Speed up = $\mathbf{1}/(\boldsymbol{S} + ((\mathbf{1}-\boldsymbol{S})/\boldsymbol{N}))$

```
Top 10 most frequent words:
1)  the - 1061396
2)  of - 593677
3)  and - 416629
4)  one - 411764
5)  in - 372201
6)  a - 325873
7)  to - 316376
8)  zero - 264975
9)  nine - 250430
10)  two - 192644
In Multithreading approach with 2 threads

Time taken to read Data file: 1.790202 seconds

Time taken to find the frequency of each word : 488.660403 seconds

Time taken to sort the top 10: 0.033099 seconds

Time taken to execute: 490.484889 seconds
```

```
Top 10 most frequent words:
1)  the - 1061396
2)  of - 593677
3)  and - 416629
4)  one - 411764
5)  in - 372201
6)  a - 325873
7)  to - 316376
8)  zero - 264975
9)  nine - 250430
10)  two - 192644
In Multithreading approach with 4 threads

Time taken to read Data file: 1.836984 seconds

Time taken to find the frequency of each word : 335.625118 seconds

Time taken to sort the top 10: 0.037814 seconds

Time taken to execute: 337.500067 seconds
```

```
Top 10 most frequent words:
1)  the - 1061396
2)  of - 593677
3)  and - 416629
4)  one - 411764
5)  in - 372201
6)  a - 325873
7)  to - 316376
8)  zero - 264975
9)  nine - 250430
10)  two - 192644
In Multithreading approach with 6 threads

Time taken to read Data file: 1.569641 seconds

Time taken to find the frequency of each word : 266.361658 seconds

Time taken to sort the top 10: 0.037331 seconds

Time taken to execute: 267.968741 seconds
```

```
Top 10 most frequent words:
1)  the - 1061396
2)  of - 593677
3)  and - 416629
4)  one - 411764
5)  in - 372201
6)  a - 325873
7)  to - 316376
8)  zero - 264975
9)  nine - 250430
10)  two - 192644
In Multithreading approach with 8 threads

Time taken to read Data file: 1.818315 seconds

Time taken to find the frequency of each word : 245.620166 seconds

Time taken to sort the top 10: 0.034772 seconds

Time taken to execute: 247.473370 seconds
```

Figure 9:Multithreading Approach Results

Number of threads	Reading Data File Time (sec)	Finding frequent words Time(parallel)sec	Sorting Time(sec)	Execution Time(sec)	Serial Time(sec)	Serial Portion%	Maximum Speed up ≈
2 threads	1.790202	488.660403	0.033099	490.484889	1.823301	0.3715	1.994
4 threads	1.836984	335.625118	0.037814	337.500067	1.874798	0.5555	3.93
6 threads	1.569641	266.361658	0.037331	267.968741	1.606972	0.5997	5.83
8 threads	1.818315	245.620166	0.034772	245.620166	1.853087	0.7544	7.598

Table 1: Multithreading calculations

The execution time is naïve was 831.900145 while here the execution time with 2 threads is 490.484889 and when we increase the number of the threads the execution time decay, this means the parallelization improves execution time, until we reach the optimal number of threads which based on system characteristics, with 8 cores the optimal number of threads is 8 any more will not improve the execution time since the overhead of managing parallel threads will increase, this overhead leads to affect the execution time in negative way.

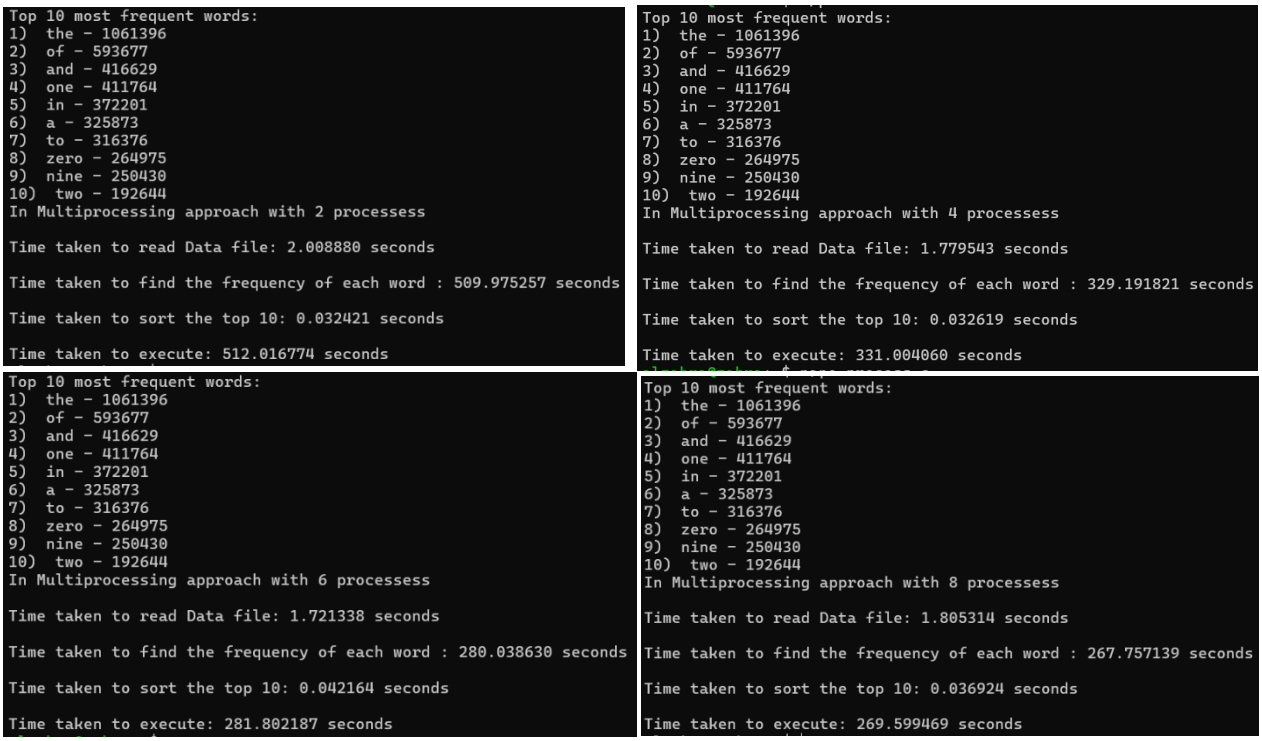
➢ **Multiprocessing Approach**

Serial Time = **Reading File Time** + **sorting the top 10 words time**

- *Note that Finding frequent words is not serial anymore*

Serial Portion = (**Serial Time** /$\boldsymbol{Total\ Time}$(execution time)) ×**100**%

Maximum Speed up = $\mathbf{1}/(\boldsymbol{S} + ((\mathbf{1}-\boldsymbol{S})/\boldsymbol{N}))$



```
Top 10 most frequent words:
1)  the - 1061396
2)  of - 593677
3)  and - 416629
4)  one - 411764
5)  in - 372201
6)  a - 325873
7)  to - 316376
8)  zero - 264975
9)  nine - 250430
10)  two - 192644
In Multiprocessing approach with 2 processess

Time taken to read Data file: 2.008880 seconds

Time taken to find the frequency of each word : 509.975257 seconds

Time taken to sort the top 10: 0.032421 seconds

Time taken to execute: 512.016774 seconds
```

```
Top 10 most frequent words:
1)  the - 1061396
2)  of - 593677
3)  and - 416629
4)  one - 411764
5)  in - 372201
6)  a - 325873
7)  to - 316376
8)  zero - 264975
9)  nine - 250430
10)  two - 192644
In Multiprocessing approach with 4 processess

Time taken to read Data file: 1.779543 seconds

Time taken to find the frequency of each word : 329.191821 seconds

Time taken to sort the top 10: 0.032619 seconds

Time taken to execute: 331.004060 seconds
```

```
Top 10 most frequent words:
1)  the - 1061396
2)  of - 593677
3)  and - 416629
4)  one - 411764
5)  in - 372201
6)  a - 325873
7)  to - 316376
8)  zero - 264975
9)  nine - 250430
10)  two - 192644
In Multiprocessing approach with 6 processess

Time taken to read Data file: 1.721338 seconds

Time taken to find the frequency of each word : 280.038630 seconds

Time taken to sort the top 10: 0.042164 seconds

Time taken to execute: 281.802187 seconds
```

```
Top 10 most frequent words:
1)  the - 1061396
2)  of - 593677
3)  and - 416629
4)  one - 411764
5)  in - 372201
6)  a - 325873
7)  to - 316376
8)  zero - 264975
9)  nine - 250430
10)  two - 192644
In Multiprocessing approach with 8 processess

Time taken to read Data file: 1.805314 seconds

Time taken to find the frequency of each word : 267.757139 seconds

Time taken to sort the top 10: 0.036924 seconds

Time taken to execute: 269.599469 seconds
```

Figure 10:Multiprocessing Approach Results

Number of processes	Reading Data File Time(sec)	Finding frequent words Time (parallel)(sec)	Sorting Time(sec)	Execution Time(sec)	Serial Time(sec)	Serial Portion%	Maximum Speed up ≈
2 processes	2.008880	509.975257	0.032421	512.016774	2.041301	0.398	1.99
4 processes	1.779543	329.191821	0.032619	331.004060	1.812162	0.547	3.93
6 processes	1.721338	280.038630	0.042164	281.802187	1.763502	0.626	5.79
8 processes	1.805314	267.757139	0.036924	269.599469	1.842238	0.682	7.65

Table 2:Multiprocessing calculations

AS multithreading, the execution time is naïve was 831.900145 while here the execution time with 2 processes is 512.016774 and when we increase the number of the processes the execution time decay, this means the parallelization improves execution time, until we reach the optimal number of child processes which based on system characteristics, with 8

cores the optimal number of child process is 8 any more will not improve the execution time since the overhead of managing parallel processes will increase, this overhead leads to affect the execution time in negative way.

1.6 Performance Measurements

Performance evaluation involves measuring the execution time and throughput of the naïve approach and multiprocessing/multithreading approach across different number of processes/threads. The effect of changing the number of processes/threads on efficiency and scalability is evaluated.

Performance (Throughput) = **1** / Execution Time

Approach	**Execution Time**	**Performance**
Naïve Approach	831.900145	0.001202
Multiprocessing with **2 processes**	512.016774	0.001953
Multiprocessing with **4 processes**	331.004060	0.003021
Multiprocessing with **6 processes**	281.802187	0.003549
Multiprocessing with **8 processes**	269.599469	0.003709
Multithreading with **2 threads**	490.484889	0.002039
Multithreading with **4 threads**	337.500067	0.002963
Multithreading with **6 threads**	267.968741	0.003732
Multithreading with **8 threads**	245.620166	0.004071

Table 3:Performance Measurements

The data shows that both multiprocessing and multithreading improve performance compared to the naïve approach, with multiprocessing generally performing better. As the number of processes or threads increases, execution time decreases, but after a certain point, the performance improvements became smaller due to overhead and resource contention. The optimal number of processes or threads appears to be around 8, as further increases don't make much of a difference.

Conclusion

After this project and after writing the codes, and understanding the work behind them, I now know how to use the parallel computing approaches multithreading, multiprocessing and naïve for efficiently finding and list the top 10 most frequent words on a large dataset, I understand the difference between these parallel computing techniques and how to create them using the APIs and the functions. I also learned how each approaches affect both execution time and performance, the results highlight the importance of choosing the right parallel computing approach and the right number of threads/processes based on the size of the dataset and system constraints.